

GISELE — Especificações Técnicas Completas

Documento de referência para desenvolvimento da plataforma do zero

Campo	Valor
Versão de referência	v2.12.1 / build 20260601-20400-bandlevels
Cliente	CPTEC/INPE — MCTI
Sigla	GISELE — Gestão Integrada de Soluções Estratégicas e Inteligência
Domínio	Visualização meteorológica e oceanográfica operacional
Documento	Especificações técnicas para reimplementação
Data	02/06/2026 (atualizado)

1. Introdução

1.1. Contexto

GISELE substitui o "SisMOM Visualizador" e existe para que pesquisadores e operadores do CPTEC/INPE consigam, na própria máquina (sem servidor de aplicação), visualizar saídas de modelos meteorológicos e oceanográficos — animar, comparar, sobrepor camadas vetoriais, extrair perfis e séries temporais, e produzir entregáveis (PNG, MP4, CSV) sem nenhuma dependência externa de pipeline.

1.2. Princípios norteadores

- 1. Zero-install para o usuário final** — abre o app e funciona. Sem stack Python/Node a instalar.
- 2. Trabalha desconectado** — uma vez baixados os dados, a animação roda offline.
- 3. Dois modos coexistentes** — produção rápida via PNG/GIF pré-renderizados do FTP do CPTEC, e modo científico via GeoTIFF decodificado em tempo real (paleta editável).
- 4. Bicópia HTML** — toda a UI vive num único arquivo .html (~1 MB) que pode ser distribuído standalone OU embalado em Electron.
- 5. Sem framework reativo** — JavaScript baunilha em IIFE. Reduzir superfície de atualização (zero-breaking) e dependências.

1.3. Escopo deste documento

Especificação completa para reimplementar o sistema a partir do zero, incluindo: stack, requisitos funcionais por módulo, requisitos não-funcionais, formato de dados, padrões de UI/UX, contratos de integração, plano de distribuição, estratégia de testes, e cronograma sugerido.

2. Visão geral do sistema

2.1. Resumo executivo

GISELE é uma **single-page application (SPA)** com renderização canvas-based:

- 4 painéis Mi (M1–M4) configuráveis em layouts 1/2/3/4 simultâneos;
- 2 abas de operação (PNG/GIF rápido vs GeoTIFF científico);
- Sistema de camadas hierárquico estilo ERMA (Background / Miscelânea / Camadas / Ferramentas);
- Decodificador GeoTIFF próprio (sem libs externas);
- Toolbox de medições e análise (distância, área, retângulo, círculo, perfil, série temporal);
- Animação com gravação MP4;
- Calculadora de raster com parser de expressão algébrica;
- Configuração persistente de modelos com placeholders de URL.

2.2. Casos de uso primários

Persona	Caso de uso	Frequência
Meteorologista de plantão	Animar precipitação Eta 3km nas próximas 72h em tela cheia, mostrando 2–4 modelos lado a lado	Diária
Pesquisador	Carregar GeoTIFF de saída de modelo, editar paleta, mascarar NoData, extrair perfil ao longo de transecto	Eventual
Oceanógrafo	Sobrepôr camada de plataformas offshore + recifes corálinos sobre TSM, gerar vídeo MP4 da evolução	Eventual
Operador	Configurar novo modelo do FTP (templates de URL com placeholders) e clonar variáveis existentes	Mensal

2.3. Não-objetivos (fora do escopo)

- Análise estatística avançada (correlações, regressões, validação cruzada);
- Edição/criação de saídas de modelo;
- Server-side rendering ou web-server hospedado;
- Autenticação/multi-tenant (acesso protegido é tratado via 2FA local opcional para modelos sensíveis);

- Mobile/touch first.

3. Stack tecnológico

3.1. Frontend (single-file SPA)

Camada	Tecnologia	Justificativa
Linguagem	JavaScript ES2020+	Suportado em todos os navegadores Chromium e Firefox modernos
Layout	HTML5 + CSS Grid + Flexbox	Sem framework — grade de painéis Mi é a única abstração pesada
Rasterização	Canvas 2D API	Universal, sem WebGL exigindo GPU
Decodificação TIFF	Decoder próprio em JS puro	Evita dependência de GeoTIFF.js e tem total controle sobre tags + NoData
Imagens cross-origin	fetch() + Blob URL	Para evitar canvas taint quando webSecurity está ativo
Cache	Memória + cache de blob URLs	LRU por (url + opts)

3.2. Empacotamento

Plataforma	Stack	Saída
Windows	Electron + electron-builder NSIS	.exe instalador + .exe portable
macOS	Electron + electron-builder dmg	.dmg Intel + .dmg Apple Silicon + .zip
Linux	Electron + AppImage	.AppImage
Standalone	HTML único	figuras_SisMOM_v23.html autosuficiente

3.3. Servidor de dados local opcional (companion)

Componente	Linguagem	Função
Python 3.8+	tools/servir_dados/servir_dados.py	Serve .png/.tif/.geojson localmente com CORS aberto
Node 18+	tools/servir_dados/servir_dados.js	Alternativa para ambientes com Node

Componente	Linguagem	Função
Bash launcher	<code>servir_dados.sh</code>	Linux/macOS
Batch launcher	<code>servir_dados.bat</code>	Windows

3.4. Documentação

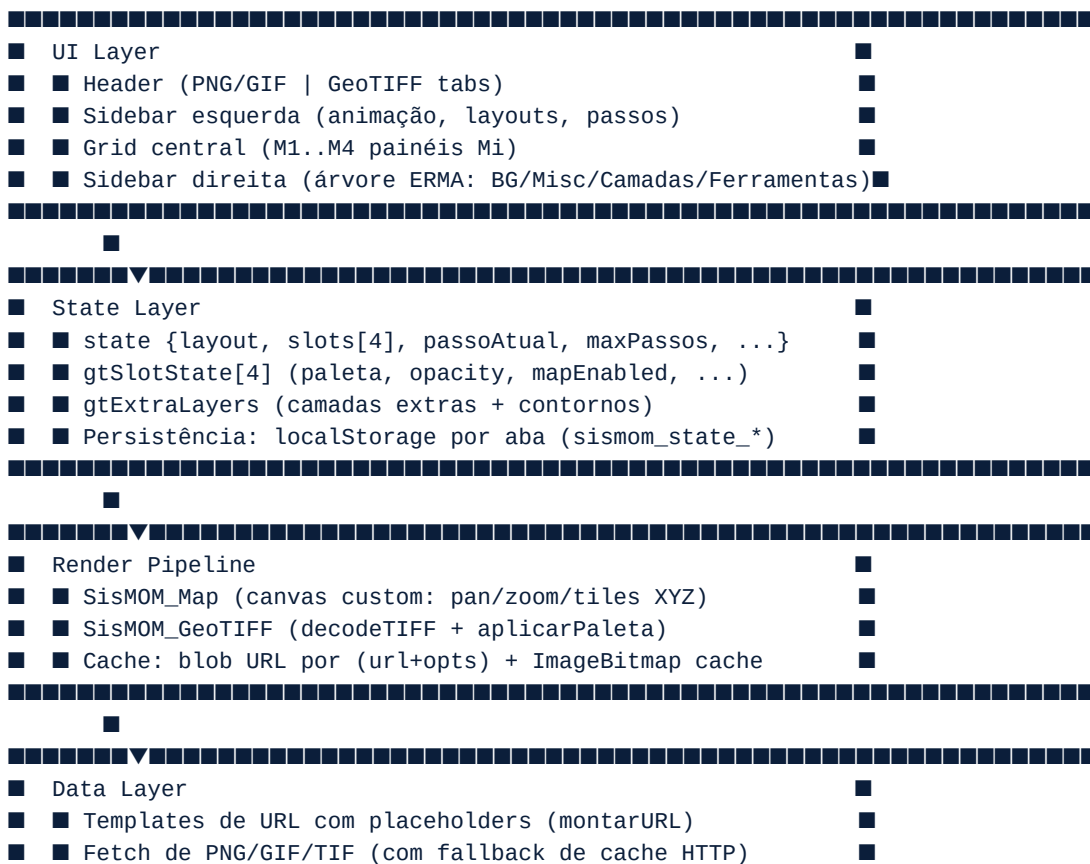
Item	Stack	Saída
Manual de uso	Python + ReportLab	docs/GISELE_Manual_Uso.pdf (~25 páginas)
Handover técnico	Markdown + pandoc + xelatex	docs/HANDOVER_GISELE.pdf

3.5. Build/CI

- npm scripts em electron-app/package.json orquestram electron-builder por plataforma;
- Hook postdist copia o HTML standalone para dist/;
- Validação: node --check no JS extraído do HTML, md5sum para verificar bicópia raiz ↔ electron-app.

4. Arquitetura de software

4.1. Diagrama lógico (camadas)



12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

#gtLayerConfigPanel é um <div> único que vive em #gtLayerConfigHome (invisível). Quando o usuário expande ■ Configuração da Camada de um nó: o panel é movido fisicamente (appendChild) para .gt-tree-config-host daquele nó. Listeners preservados. Apenas UM expandido por vez.

5. Requisitos funcionais (RF)

5.1. RF-01: Aba PNG/GIF — animação rápida

- **RF-01.1:** Mostrar imagens PNG/GIF do FTP CPTEC em até 4 painéis simultâneos
- **RF-01.2:** Animação play/pause/stop com velocidade 0.2/0.5/1/2 s
- **RF-01.3:** Setas ←/→ navegam passo a passo; Espaço alterna play/pause; Esc cancela ferramenta
- **RF-01.4:** Pré-carregamento (prefetch) dos próximos passos durante animação
- **RF-01.5:** Cache de blob URL para evitar refetch na 2ª volta da animação
- **RF-01.6:** Zoom via roda do mouse, pan via clique-arrasto, reset via duplo-clique ou botão

5.2. RF-02: Aba GeoTIFF — visualização científica

- **RF-02.1:** Decodificar TIFF baseline (8/16/32/64-bit, Float/Int) com geo-tags
- **RF-02.2:** Aplicar paletas (mínimo 16: viridis, plasma, inferno, magma, cividis, jet, turbo, gray, rdbu, rdylbu, spectral, brbg, seismic, coolwarm, terrain, ocean)
- **RF-02.3:** Min/max auto via percentil + edição manual
- **RF-02.4:** Mascaramento NoData manual (lista de valores) e clip $\geq/\leq$
- **RF-02.5:** Renderização scanline (256 strips) em Mercator para preservar projeção
- **RF-02.6:** Mapa-base XYZ tiles configurável: Esri World Imagery, OSM, OpenTopoMap, ou nenhum

5.3. RF-03: Multi-painel Mi

- **RF-03.1:** 4 slots (M1, M2, M3, M4) com layouts 1/2/3/4 painéis
- **RF-03.2:** Cada slot tem seu próprio modelo + variável + data + estado de zoom
- **RF-03.3:** Sincronização opcional de data com M1 (botão)
- **RF-03.4:** Painel ATIVO recebe edições do sidebar direito (Mi-ativo é destacado)
- **RF-03.5:** HUD por slot mostra zoom + lat/lon do cursor + valor sob o cursor

5.4. RF-04: Sistema de camadas (árvore ERMA)

- **RF-04.1:** 4 grupos colapsáveis no painel direito: Background, [PIN] Miscelânea, ■ Camadas, [TOOL] Ferramentas
- **RF-04.2:** Botão "Collapse folders / Expand folders" (toggle global)
- **RF-04.3:** Background: radios mutuamente exclusivos para tile provider do slot ativo

- **RF-04.4:** Miscelânea: checkboxes que adicionam/removem camadas pré-empacotadas
- **RF-04.5:** Camadas: lista live de todas (primary + extras + contornos + geojson) com:
 - Checkbox de visibilidade
 - Nome (clique ativa a camada)
 - Color picker (geojson) ou paleta (no sub-menu)
 - Botão × (remover)
 - Sub-menu ■ Configuração da Camada (apenas uma por vez)
- **RF-04.6:** Ferramentas: botões para Adicionar GeoTIFF/GeoJSON, Adicionar Modelo, Abrir TIF local, e sub-menu [NUM] Calculadora

5.5. RF-05: Configuração da Camada (sub-menu por camada)

Disponível em cada camada raster, contém em layout vertical:

- Paleta (16 opções)
- Min/Max + Editar escala (auto vs manual)
- UNDEF: lista de NoData manuais
- Clip $\geq$ e Clip $\leq$
- Contornos: enabled, modo (auto/manual), N níveis, lista de níveis, cor, largura, rotular cada N, manter preenchimento
- Colorbar visual
- Info (dim + bbox + nodata)
- Inverter Y (botão manual)
- [NUM] Calc: operador + escalar + Aplicar → cria nova camada (camada OP scalar)

5.6. RF-06: Ferramentas de medição e análise

Ícone	Ferramenta	Especificação
	Distância	Polilinha, soma de segmentos via Haversine, mostra km
■	Área	Polígono fechado, área esférica, mostra km ²
■	Retângulo	2 cliques (canto opostos), área esférica
■	Círculo	Centro + raio (clique-arrasto), raio em km + área
■	Linha	Polilinha de anotação (sem medição)

Ícone	Ferramenta	Especificação
T	Texto	Rótulo no mapa em ponto clicado
■	Perfil	Polilinha, amostra camada ATIVA a cada pixel, gráfico tempo×distância
■	Série temporal	1 ponto, varre todos os passos do slot, gráfico tempo×valor

Cada ferramenta:

- Ativa via clique no ícone na toolbar do mapa
- Vértices via cliques individuais; duplo-clique finaliza
- Esc cancela; volta automaticamente para Pan ao finalizar
- Wheel zoom + pan via arrasto seguem funcionando durante o desenho

5.7. RF-07: Calculadora de raster

Per-layer (em Configuração da Camada):

- Operador + - × ÷ + escalar → cria nova camada = camada_atual OP scalar
- Erro se camada não tem dados decodificados ou escalar inválido

Inter-layer (em Ferramentas → [NUM] Calculadora):

- Lista de tokens clicáveis (Camada1, Camada2, ...) que inserem na expressão
- Textarea para expressão
- Parser próprio que suporta números (com expoente), identificadores, + - * / × ÷ (), parênteses, unários
- Recusa qualquer outro símbolo (sem code injection)
- Per-pixel: avalia AST com vars = valores das camadas referidas
- Máscara propaga: qualquer NoData operando → NoData resultado
- Validação: todas as camadas referidas precisam ter mesma dimensão

5.8. RF-08: Miscelâneas (camadas vetoriais)

- **RF-08.1:** Diretório miscelaneas/ com manifest.json + arquivos .geojson
- **RF-08.2:** Manifest + GeoJSONs embutidos como `<script type="application/json">` no final do HTML (funciona em file://)
- **RF-08.3:** Cada item tem id, nome, arquivo, labelProp, infoProps[], style{}, fonte
- **RF-08.4:** Style suporta: stroke, fill, fillColor, linewidth, pointRadius, hatch, hatchColor, hatchSpacing, hatchLineWidth
- **RF-08.5:** Hachura diagonal via CanvasPattern com cache local por (cor, spacing, larguraLinha)

- **RF-08.6:** Color picker no chip muda stroke + hachura + fill rgba preservando alpha
- **RF-08.7:** Clique em shape (modo Pan): point-in-polygon real para Polygon/MultiPolygon (respeita buracos), tolerância ~10 px para Point → popup branco com tabela de infoProps
- **RF-08.8:** Datasets entregues:
 - `plataformas_offshore_brasil.geojson` (107 pontos rotulados — plataformas offshore)
 - `corais_brasil.geojson` (11 polígonos — recifes coralinos da costa brasileira, derivados do shapefile WCMC008)

5.9. RF-09: Configuração de modelos

- **RF-09.1:** Modal "Configurar" com abas por modelo
- **RF-09.2:** CRUD: novo, clonar, remover, restaurar padrão, exportar/importar JSON
- **RF-09.3:** Campos: nome, subsistema, escopo1/2, sufixo arquivo (.png/.gif/.tif), maxPassos, template URL PNG, template nome PNG, formatos disponíveis (PNG/TIF), templates TIF (próprios ou herdados de PNG)
- **RF-09.4:** Mapa-base padrão (none/esri/osm/topo) — aplicado quando modelo é selecionado em slot
- **RF-09.5:** Botão "Preset FTP CPTEC": preenche templates assumindo padrão CPTEC (PNG em /fig/, TIF em /geotiff/)
- **RF-09.6:** Tabela de variáveis por modelo: id, label, unidade, frequência (h), horizonte (h), prefixo, checkboxes disp_png/disp_tif
- **RF-09.7:** Persistência: localStorage chave `sismom_models` (override sobre DEFAULT_MODELOS)

5.10. RF-10: Templates de URL (placeholders)

Placeholder	Substituição
{yyyy}, {mm}, {dd}, {hh}	Data da rodada (caminho) ou validade (nome do arquivo)
{N}, {N%4}	Índice do arquivo = $\text{Math.round}(\text{passo} / \text{freq})$ com N casas
{F}, {F%3}	Hora de previsão = $\text{file_idx} \times \text{freq}$ com N casas
{prefixo}	Campo "arquivo" da variável
{escopo1}, {escopo2}	Tokens livres do modelo/variável
{ext}	Extensão (.png, .tif, etc.)
{data}	Data da rodada formato YYYYMMDDHH
{f%N}	Atalho f + horas zero-padded N casas

5.11. RF-11: Salvar vídeo MP4

- **RF-11.1:** Botão "Salvar vídeo (MP4)" abaixo do seletor de velocidade
- **RF-11.2:** Disponível em ambas as abas (PNG/GIF e GeoTIFF)

- **RF-11.3:** Passagem **única** do primeiro ao último passo (sem looping)
- **RF-11.4:** Fase 1 (pré-busca): paraleliza fetch de todos os passos como blob URLs
- **RF-11.5:** Fase 2 (gravação): captura via MediaRecorder a 30 fps em codec MP4 (com fallback WebM)
- **RF-11.6:** Preserva área visível (zoom/pan) — em PNG usa source crop por object-fit; em GeoTIFF captura o canvas direto
- **RF-11.7:** Cada passo dura ≥ 400 ms no vídeo (frame estável)
- **RF-11.8:** Popup com progresso (Pré-buscando X/Y → Gravando X/Y) + botão Cancelar
- **RF-11.9:** Nome do arquivo: `evolucao_<modelo>_<variavel>_<timestamp>.<mp4|webm>`

5.12. RF-12: Servidor HTTP local opcional

- **RF-12.1:** Scripts Python e Node em `tools/servir_dados/`
- **RF-12.2:** Serve `.png/.gif/.tif/.geojson` com headers CORS abertos
- **RF-12.3:** MIME corretos (incluindo `image/tiff`)
- **RF-12.4:** Proteção path traversal
- **RF-12.5:** Launchers para Linux (`.sh`), Windows (`.bat`), macOS

5.13. RF-13: Distribuição

- **RF-13.1:** Build Windows NSIS installer + portable exe
- **RF-13.2:** Build macOS `.dmg` para Intel + Apple Silicon, + `.zip` alternativo
- **RF-13.3:** Build Linux AppImage
- **RF-13.4:** HTML standalone sempre incluído no `dist/` via hook `postdist`
- **RF-13.5:** Suporte CLI flags:
 - `--displays=1,2,5,6` → spanning multi-monitor
 - `--all-displays` → cobre todos os monitores
 - `--no-frame` → kiosk mode sem frame de janela
 - `--strict-cors` → ativa webSecurity (modo seguro, mas quebra vídeo MP4 PNG)
- **RF-13.6:** Atalho de teclado F11 (fullscreen toggle) + Ctrl+Q (sair)

5.14. RF-14: Monitoramento — rotas de dados genéricos (v2.12)

Fonte de dados **independente dos modelos**: rotas **KML/GeoJSON** configuradas na aba *Base de dados* (rota, template de nome, sufixo, mapa-base, "sempre trazer dados atualizados"). Menu **[ANT] Monitoramento** na árvore ERMA, uma linha por base (liga/desliga, cor,  **atualizar** com `cache:'no-store'`, filtro Todas/Ativas/Inativas, remover, reordenar). Parser KML embarcado (DOMParser → Point/LineString/Polygon → GeoJSON; atributos extraídos da tabela HTML do `<description>`). Seed de fábrica: **Queimadas recentes (INPE)** — marcador (path vetorial, não emoji), rótulo acima do ponto, foco ativo×apagado por cor do miolo e popup read-only de atributos. Config

persistível em arquivo (%APPDATA%/GISELE/configuração/gisele-config.json no Electron).

5.15. RF-15: Calculadora Temporal (v2.6)

Na Configuração da Camada, expressões entre tempos da mesma rodada: sintaxe t_N/h_N , ranges $t1..t24$, funções `sum/mean/max/min/count`. Gera camada derivada; modal de progresso com fetches paralelos.

5.16. RF-16: Exportar/Importar dados vetoriais (v2.6–v2.8)

Exportar o campo inteiro ou um recorte (polígono/retângulo/por camada/área total) como **GeoJSON** (nuvem de pontos) com `metadata.stats` (min, max, soma, média, média ponderada por área, área total) e popup de confirmação. Exportar **série temporal de um ponto** → GeoJSON. Importar **Shapefile** (.shp ou .zip, parser JS puro + ZIP via `DecompressionStream`) e GeoJSON como camada vetorial, com preview e diálogo de confirmação.

5.17. RF-17: Série temporal e perfil interativos (v2.x)

Perfil ao longo de polilinha e **série temporal num ponto** (eixo X tempo, eixo Y valor). Gráficos interativos: toggle por chip da legenda, zoom por click-and-drag (rubber-band), tooltip multi-série, exportação CSV e PNG. Multi-painel (curvas por painel) e multi-camada (uma curva por camada raster ativa; camadas de cálculo reavaliadas **por tempo de validade**).

5.18. RF-18: Polígonos do usuário (v2.9)

Storage `gisele.savedPolygons.v1` (localStorage; disco no Electron) com `save/list/rename/setColor/remove`. Submenu em Ferramentas: desenhar e salvar, exportar/importar .geojson, visualizar (toggle), gerenciar (perímetro/área/bbox) e usar como **máscara de recorte** no Exportar GeoJSON.

5.19. RF-19: Sincronização multi-painel (v2.9)

API `getViewport/applyViewportRaw/listener` no `SisMOM_Map`: propagação de viewport entre painéis, **lock por painel** () com replicação de anotações (distância/linha/texto/perfil), **perfil combinado** e **série temporal multi-painel** (amostragem paralela, CSV combinado). Painéis de análise (`frequência=0`, ex. MERGE) recuam a data automaticamente até a observação disponível.

5.20. RF-20: Helper Python local (v2.7) + cliente `gisele_ts` (v2.10)

Subprocess opcional embarcado no Electron (FastAPI + rasterio + httpx): acelera extração temporal, calculadora temporal e perfil com fetches paralelos (~10×), cache decoded em memória e endpoint de render PNG server-side (matplotlib). **Fallback transparente para JS** quando offline (badge ■/JS). Cliente Python standalone `api-client/gisele_ts` envelope `/v1/timeseries/point` para uso em scripts/notebooks.

5.21. RF-21: Anotação e desenho (v2.11)

No **modo PNG/GIF**: linha/área/texto em coords normalizadas à imagem, com seleção, **lock e replicação por painel**. **Caneta de tela** global (canvas position:fixed acima de tudo): paleta, espessura, desfazer/limpar, independente de modo.

5.22. RF-22: Render do raster GeoTIFF (v2.12.1)

Raster interpolado (bilinear, imageSmoothingEnabled) com modos de sombreado **Suavizado | Bandas | Pixel** (gisele.raster.mode). No modo **Bandas**, as faixas de cor seguem exatamente os **mesmos níveis do contorno** (custom ou N auto), via setBandLevels; contorno e shaded compartilham os níveis.

6. Requisitos não-funcionais (RNF)

6.1. Performance

Métrica	Alvo
Tempo de carregamento inicial	< 2 s em Electron, < 3 s em standalone
Decodificação TIFF típico (1000×1000 Float32)	< 200 ms
Aplicação de paleta	< 50 ms
Animação a 5 fps (200 ms/frame)	Sem queda perceptível em modelos típicos
Memória residente	< 500 MB com 4 painéis × 120 passos cacheados

6.2. Compatibilidade

- **Browsers**: Chrome/Edge 90+, Firefox 90+, Safari 14+ para standalone
- **OS empacotado**: Windows 10/11, macOS 11+, Ubuntu 20.04+
- **Resolução mínima**: 1024×768; suporta multi-monitor até 8 displays

6.3. Acessibilidade

- ARIA labels em controles principais (transport buttons, tabs, sliders)
- Navegação via teclado em formulários
- Contraste WCAG AA mínimo para textos importantes
- aria-live para anúncios de mudança de estado (toasts)

6.4. Internacionalização

- UI em português (pt-BR) na v1.x
- Strings centralizadas para facilitar i18n futura
- Datas em UTC com formato dd/mm/yyyy + 00Z notation

6.5. Segurança

- `--strict-cors` opt-in para reativar same-origin policy
- Parser de expressão (Calculadora) recusa qualquer caractere não-permitido (sem `eval`)
- Configurações sensíveis (`requires2FA`) gateadas por TOTP local
- Imagens PNG/GIF cross-origin tratadas via blob fetch para evitar canvas taint

6.6. Robustez

- Toda I/O com try/catch e fallback gracioso
- Tail HTML validado a cada build (`</html>` final + JSON inline parseável)
- `node --check` no JS extraído como gate pre-commit
- Recuperação de localStorage corrompido (parse com fallback para defaults)

6.7. Observabilidade

- Console log `[GISELE] build = YYYYMMDD-NNNN-name` a cada boot
 - Logs categorizados: `[GISELE/TIFF]`, `[GISELE/Profile]`, `[GISELE/Click]`, `[GISELE/TS]`, etc.
 - `launch.log` no Electron (`%APPDATA%/GISELE/launch.log`) com versões + CORS mode + bounds dos displays
-

7. Modelo de dados

7.1. Estado global `state`

```
{
  layout: 1,                // 1..4 painéis simultâneos
  passoAtual: 1,            // passo atual da animação (horas)
  maxPassos: 120,           // max(v.horizonte) entre slots ativos
  stepFreq: 1,              // min(v.frequencia) entre slots ativos
  tempo: 200,               // ms por frame na animação
  animando: false,
  interval: null,           // handle do setInterval
  slots: [
    { modelo, variavel, data, sync, passoBase, lastVarByModel }, // 4 slots
    ...
  ]
}
```

7.2. Estado por slot `GeoTIFF` `gtSlotState[i]`

```
{
  paleta: 'viridis',
  autoMinMax: true,
  min: null, max: null,
  undefRaw: '',             // lista CSV de NoData manuais
  clipBelow: null, clipAbove: null,
}
```

```

mapEnabled: false,
mapProvider: 'esri',
opacity: 0.85,
_lastModelForMap: null,          // flags internos para auto-aplicar mapProvider
_mapProviderUserSet: false,
_mapEnabledUserSet: false
}

```

7.3. Camada (layer)

```

{
  id: 'primary' | 'ext_*' | 'calc_*' | 'ctr_*',
  type: 'geotiff' | 'geojson' | 'contour',
  name: 'string visível',
  visible: true,
  opacity: 0.85,
  decoded: { width, height, data: Float32Array, bbox, min, max, nodata, nodataExtras, scale } | null
  paleta: 'viridis',
  color: '#0aa37a',              // geojson only
  isMisc: true,                  // miscelânea
  miscConfig: { /* item do manifest */ },
  data: { /* GeoJSON FeatureCollection */ }, // geojson only
  props: {                       // geotiff only
    paleta, autoMinMax, customMin, customMax,
    undefRaw, clipBelow, clipAbove,
    contours: { enabled, mode, count, levels, color, lineWidth, labelEvery, keepFill }
  }
}

```

7.4. Modelo

```

{
  nome: 'Regional · Eta 3km',
  subsistema: 'atmos' | 'ocean',
  maxPassos: 120,
  extensao: '.png',
  freq_rodadas: 6,
  url_path: 'https://.../{yyyy}/{mm}/{dd}/{hh}/fig',
  file_name: '{prefixo}-{F%3}{ext}',
  tem_png: true, tem_tif: false,
  extensao_tif: '.tif',
  same_url_for_tif: false,
  url_path_tif: '...',
  file_name_tif: '...',
  same_name_for_tif: false,
  mapProvider: 'esri',          // default basemap quando modelo carrega
  requires2FA: false,
  escopo1: '', escopo2: '',
  variaveis: [
    {
      id: 'PREC', label: 'Precipitação horária', unidade: 'mm',
      frequencia: 1, horizonte: 120, arquivo: 'prec_eta3km_SisMOMoper',
      escopo1: '', escopo2: '',
      disp_png: true, disp_tif: false
    },
    ...
  ]
}

```

7.5. Decoded TIFF

```
{
  width: 1000, height: 800,
  data: Float32Array(800000),
  bbox: { minX, minY, maxX, maxY },
  scale: { sx, sy },
  nodata: -9999 | null,
  nodataExtras: [1e20, -1e35] | null,
  min: 0.0, max: 1023.5,
  pixelIsPoint: false           // se vier de GTRasterTypeGeoKey=1
}
```

7.6. Manifest de miscelâneas

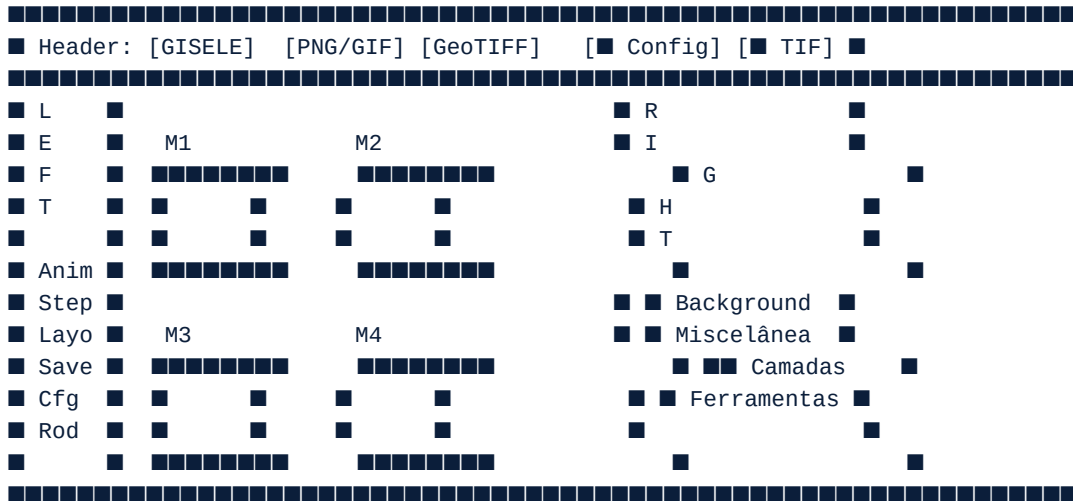
```
{
  "version": 1,
  "description": "...",
  "items": [
    {
      "id": "plataformas_br",
      "nome": "Plataformas offshore (Brasil)",
      "arquivo": "plataformas_offshore_brasil.geojson",
      "tipo": "geojson",
      "labelProp": "nome",
      "infoProps": ["nome", "operadora", "campo", ...],
      "style": {
        "stroke": "#ff7a00",
        "fill": "rgba(34,200,154,0.18)",
        "fillColor": "#ff7a00",
        "lineWidth": 1.2,
        "pointRadius": 5,
        "hatch": true,
        "hatchColor": "#0aa37a",
        "hatchSpacing": 7,
        "hatchLineWidth": 1
      },
      "fonte": "Citação da fonte original"
    }
  ]
}
```

7.7. Persistência (localStorage keys)

Chave	Conteúdo
sismom_app_mode	'png' ou 'gtiff' (sempre forçado 'png' no boot)
sismom_state_png	snapshot do state em modo PNG
sismom_state_gtiff	snapshot do state em modo GeoTIFF
sismom_models	modelos editados (override sobre defaults)
sismom_ui_prefs	preferências de UI (layout, tempo, passoAtual)
sismom_two_factor_*	seeds 2FA quando relevante

8. Interface do Usuário

8.1. Layout principal



8.2. Padrões de UI

- **Cores:** paleta dark (#0b1220 base, accent ciano #4dd0e1, accent emerald #10b981)
- **Tipografia:** ui-sans-serif system-ui (UI), ui-monospace (números, código)
- **Bordas:** 1px solid + 1px dashed para separadores
- **Disclosure** (<details>): chevron customizado (rotaciona 90° quando aberto)
- **Botões:** ghost (transparent + border), primary (azul), danger (vermelho)
- **Inputs:** background elevado, border soft, focus border accent
- **HUD overlay:** posição absolute, fundo semitransparente (rgba 0,0,0,0.55)

8.3. Padrões de árvore (ERMA-style)

- Grupo aberto por padrão (<details open>)
- Botão global "Collapse folders / Expand folders" (toggle inteligente)
- Apenas 1 sub-config aberto por vez (Configuração da Camada)
- Layout vertical dentro dos hosts (não horizontal-wrap)

9. APIs e Integrações

9.1. APIs internas (módulos JS)

Módulo	API Pública
SisMOM_GeoTIFF	decodeTIFF(buffer), aplicarPaleta(decoded, opts), GT_PALETTES, isGeoTiffModel(m)
SisMOM_Map	new SisMOM_Map(canvas, opts) → setTileProvider, setProjection, fitTo(bbox), setRasterOverlay, addGeoJSON, setAttributionElement, onCursor, zoomBy, redraw
Calculadora	gtParseExpr(src), gtCollectIdents(ast), gtEvalAst(ast, vars), gtCreateLayerFromExpression(expr, nameToLayer, displayName)
Pipeline	montarURL({modelo, data, variavel, passo}), carregarImagem(slotIdx, url), gtDeriveTifUrl(url)

9.2. Integrações externas (read-only)

- **FTP do CPTEC:** HTTPS GET de .png/.gif/.tif (servidor não envia CORS — Electron contorna via webSecurity:false; standalone precisa de servidor proxy local)
- **Esri World Imagery:** tiles XYZ (atribuição obrigatória)
- **OpenStreetMap:** tiles XYZ (atribuição obrigatória)
- **OpenTopoMap:** tiles XYZ (atribuição CC-BY-SA)

9.3. CLI Flags (Electron)

Flag	Efeito
--displays=1,2,5,6	Estende janela spanning monitors específicos
--all-displays	Cobre todos os monitores detectados
--no-frame	Modo kiosk sem frame
--strict-cors	Reativa webSecurity (segurança alta; vídeo MP4 PNG falha)

10. Distribuição e Deploy

10.1. Estrutura de release

```

GISELE-2.3.0/
■■■ GISELE Setup 2.3.0.exe           # Windows NSIS
■■■ GISELE-2.3.0-portable.exe        # Windows portable
■■■ GISELE-2.3.0.dmg                 # macOS Intel
■■■ GISELE-2.3.0-arm64.dmg           # macOS Apple Silicon
■■■ GISELE-2.3.0.AppImage            # Linux AppImage

```

```

■■■■ GISELE-2.3.0-standalone.zip      # HTML único + miscelaneas/
■■■■ docs/
■    ■■■■ GISELE_Manual_Uso.pdf
■    ■■■■ HANDOVER_GISELE.pdf
■■■■ tools/servir_dados/              # Servidor HTTP local opcional
    ■■■■ servir_dados.py
    ■■■■ servir_dados.js
    ■■■■ servir_dados.sh
    ■■■■ servir_dados.bat

```

10.2. Pipeline de build

```

cd electron-app/
npm install
npm run dist:win      # → .exe + portable
npm run dist:mac      # → .dmg + .zip (Intel + ARM)
npm run dist:linux    # → .AppImage
npm run postdist      # → copia HTML standalone para dist/

```

Hook postdist em package.json:

```

"scripts": {
  "postdist": "node -e \"require('fs').copyFileSync('../figuras_SisMOM_v23.html', 'dist/GISELE-standalone.html')\"
}

```

10.3. Atalhos de instalação

- **Linux:** instalar-atalho.sh cria ~/.local/share/applications/GISELE.desktop
- **Windows:** usuário cria atalho manual (ou via Setup)
- **macOS:** drag-and-drop padrão do .dmg

10.4. Versionamento

- SemVer estrito: MAJOR.MINOR.PATCH
- Build markers internos: YYYYMMDD-NNNN-nome (logs)
- Tags git: v2.3.0

11. Estratégia de testes

11.1. Testes manuais (smoke tests por release)

1. **Boot:** abrir em Electron (Win/Mac/Linux) e em standalone → console mostra build marker
2. **PNG/GIF:** carregar Eta 3km → animar 30 passos → save MP4 → conferir vídeo
3. **GeoTIFF:** abrir TIF local → trocar paletas → editar min/max → ajustar NoData → ativar contornos
4. **Multi-painel:** layout 4 → 4 modelos diferentes → animar todos sincronizados
5. **Miscelâneas:** ativar Plataformas + Corais → clicar em shape → popup info → trocar cor

6. **Ferramentas:** cada uma das 8 ferramentas (distância, área, etc.)
7. **Calculadora:** per-layer × scalar + expressão entre 2 camadas
8. **Swap PNG↔GeoTIFF:** trocar 5x → estado preservado por aba

11.2. Testes automatizados

11.2.1. JS syntax (pre-commit gate)

```
import re, subprocess, tempfile, os
html = open('figuras_SisMOM_v23.html').read()
scripts = re.findall(r'<script(?:[^\>]*type=["\']application/json["\'])(?:[^\>]*)>([\s\S]*)')
combined = '\n\n'.join(s for s in scripts if s.strip())
with tempfile.NamedTemporaryFile(mode='w', suffix='.js', delete=False) as f:
    f.write(combined); tmp = f.name
r = subprocess.run(['node', '--check', tmp], capture_output=True, text=True)
assert r.returncode == 0, r.stderr
```

11.2.2. Bicópia md5 idêntica

```
md5sum figuras_SisMOM_v23.html electron-app/figuras_SisMOM_v23.html
# As duas linhas de hash DEVEM ser idênticas
```

11.2.3. JSON inline parseável

```
import json, re
html = open('figuras_SisMOM_v23.html').read()
for tag_id in ['gt-misc-manifest', 'gt-misc-data-plataformas_br', 'gt-misc-data-corais_br']:
    m = re.search(r'<script type="application/json" id="' + tag_id + r'">([\s\S]*)</script>')
    assert m, f"{tag_id} not found"
    json.loads(m.group(1).strip()) # raises se inválido
```

11.2.4. Smoke test do decoder (Node)

Gerar um TIFF sintético 32×32 Float32 com bbox conhecido → executar `SisMOM_GeoTIFF.decodeTIFF(buffer)` extraído do HTML em Node → verificar bbox/min/max + aplicar 5 paletas.

11.3. Verificações pré-release

- [] node --check passa nos 2 HTMLs
- [] md5sum idênticos
- [] JSON inline OK (3 tags)
- [] Tail termina com `</script>\n</body>\n</html>\n`
- [] Build marker bumped
- [] Manual PDF regenerado
- [] HANDOVER PDF atualizado
- [] Smoke test do decoder OK
- [] Builds gerados para 3 plataformas

- [] Standalone .zip contém HTML + miscelaneas/

12. Cronograma sugerido

12.1. Fases (desenvolvimento do zero)

Fase	Duração	Entregáveis
Fase 0: Estudo + setup	2 semanas	Repo inicial, build pipeline Win/Mac/Linux, CI
Fase 1: Pipeline PNG/GIF	4 semanas	Aba PNG, multi-painel, animação, sidebar esquerda
Fase 2: Decoder + paletas	3 semanas	SisMOM_GeoTIFF.decodeTIFF, 16 paletas, modal de visualização
Fase 3: Mapa custom + tiles	3 semanas	SisMOM_Map, projeção Mercator/PlateCarree, tiles XYZ, pan/zoom
Fase 4: Multi-Mi GeoTIFF	4 semanas	gtSlotState, sidebar direita (acordion), camada ativa, dispatch
Fase 5: Camadas extras + Calc	2 semanas	gtExtraLayers, calculadora binária, contornos marching squares
Fase 6: Ferramentas	3 semanas	Distância, área, rect, círculo, linha, texto, perfil, série temporal
Fase 7: Miscelâneas	2 semanas	Manifest, embed inline, hachura, color picker, popup info
Fase 8: Vídeo MP4	1 semana	MediaRecorder, pré-busca, holdAndPaint, codec fallback
Fase 9: Configuração avançada	2 semanas	CRUD modelos, presets, mapProvider, 2FA opcional
Fase 10: Reorganização UI (árvore ERMA)	2 semanas	4 grupos, Configuração da Camada móvel, Collapse folders
Fase 11: Calculadora avançada	1 semana	Parser de expressão, per-pixel evaluator, integração dupla
Fase 12: Distribuição + Documentação	2 semanas	Electron Win/Mac/Linux, standalone, manual PDF, HANDOVER

Fase	Duração	Entregáveis
Fase 13: Hardening + testes	2 semanas	Smoke tests automatizados, fixes finais, beta

Total estimado: ~33 semanas (~7,5 meses) com 1 dev sênior + 1 pleno.

12.2. Equipe mínima sugerida

- 1× **Tech lead** (full-stack JS sênior, com background gráfico/cartográfico)
- 1× **Dev pleno** (JS + Python para servidor local + scripts)
- 1× **Designer/UX** part-time (mockups, paletas, ícones SVG)
- 1× **Meteorologista** (consultor, define corretude científica das paletas, NoData, perfis)

12.3. Marcos críticos

- M1 (fim Fase 3): demonstrar decoder + paleta + mapa renderizando 1 TIFF
- M2 (fim Fase 6): demonstrar perfil + série temporal em modelo real
- M3 (fim Fase 10): UX final aprovada pela equipe meteorológica
- M4 (fim Fase 13): release v1.0.0

13. Riscos e mitigações

Risco	Impacto	Mitigação
FTP CPTEC mudar estrutura de URL	Alto — quebra todos os modelos	Templates configuráveis + 2FA opt-in + presets
Browser/Electron lança canvas-taint mesmo com webSecurity:false	Médio — vídeo PNG falha	Fallback via blob fetch + --strict-cors opt-in com docs
TIFF com compressão LZW exótica	Médio — não decoda	Embutir lib UTIF como fallback opcional
Modelos legacy com maxPassos obsoleto	Baixo — UI quebra ao trocar	Snap obrigatório em atualizarMaxPassos, fallback v.horizonte
Performance ruim em modelo com >2000×2000 pixels	Médio — animação trava	Web Workers para decode + ImageBitmap cache
Bicópia raiz/electron divergir	Alto — bug no Electron sem aparecer no standalone	CI gate em md5sum

Risco	Impacto	Mitigação
Tail HTML truncar em edits massivos	Crítico — quebra browser	Validação automática <code></html></code> + reconstrução do <code>anchor</code> <code>gt-misc-data-corais_br</code>
Loop infinito em event handlers (toggle/render)	Crítico — congela UI	Padrão: <code>e.preventDefault()</code> + <code>flags</code> <code>_userSet</code> + <code>click-on-summary</code> em vez de <code>toggle</code>

14. Apêndices

14.1. Glossário

Termo	Definição
Painel Mi	Um dos 4 quadrantes M1/M2/M3/M4 no grid central
Slot	Estado de um painel Mi (modelo + variável + data + zoom)
Camada Primary	A camada principal de um slot (raster do modelo)
Camada Extra	Sobreposição adicional (raster ou vetor)
Rodada	Run do modelo (data + hora) — define o caminho na URL
Passo	<code>Forecast hour</code> — <code>passo_h = file_idx × frequencia</code>
Validade	Data/hora prevista = rodada + <code>passo_h</code>
NoData / Sentinel	Valor representando ausência de dado (ex: -9999, 1e20)
Bbox	Bounding box {minX, minY, maxX, maxY} em lat/lon

14.2. Convenções de código

- Funções públicas: `camelCase` (e.g. `gtRenderTreeLayers`)
- Variáveis privadas/internas: `underscorePrefix` (e.g. `_gtCfgPanel`)
- Constantes: `UPPER_SNAKE` (e.g. `GT_TOOL_COLOR`)
- IDs DOM: `gt` prefix para GeoTIFF, `sismom` ou puro para legacy

14.3. Histórico de versões da plataforma de referência

Versão	Build marker	Highlights
v1.0	(sem marker)	SisMOM Visualizador legacy (PNG/GIF only)
v2.0	20260528-XXXX-gisele	Rebrand para GISELE + GeoTIFF + multi-Mi
v2.1	20260528-4600-notoggle	Ferramentas + Miscelâneas (plataformas + corais)
v2.2	20260529-1900-videofitfps	Série temporal + vídeo MP4 + mapa default + fixes
v2.3	20260529-3000-calc	Árvore ERMA + Configuração da Camada + Calculadora dupla

Especificação compilada em 29/05/2026 para uso em reimplementação. Refer to [HANDOVER_GISELE.md/.pdf](#) para detalhes de implementação da versão de referência.